

Assignment 1

1. Aim

To study unsafe data type conversions in C++ and implement safe conversion mechanisms using range checks and validation.

2. Objectives

- Demonstrate int to short truncation.
- Demonstrate signed-to-unsigned conversion.
- Demonstrate floating-point to integer truncation.
- Demonstrate signed integer overflow.
- Demonstrate string-to-numeric conversion.
- Implement range validation.
- Compare unsafe and safe behavior.

3. Methodology

Two C++ programs were developed. The first program performs type conversions without validating the input range. The second program introduces validation functions that check whether the source value can be represented by the destination type before conversion.

4. Unsafe Implementation

```
#include <iostream>
#include <string>
#include <limits>

using namespace std;

int main() {
    // 1. int -> short : truncation
```

```
int largeInt = 40000;
short smallShort = (short)largeInt;

cout << "INT -> SHORT\n";
cout << "Before: " << largeInt << endl;
cout << "After : " << smallShort << endl;

// 2. signed -> unsigned : sign conversion
int negative = -25;
unsigned int positive = (unsigned int)negative;

cout << "\nSIGNED -> UNSIGNED\n";
cout << "Before: " << negative << endl;
cout << "After : " << positive << endl;

// 3. float -> int : fractional truncation
float decimal = 123.78f;
int number = (int)decimal;

cout << "\nFLOAT -> INT\n";
cout << "Before: " << decimal << endl;
cout << "After : " << number << endl;

// 4. Integer overflow
int maxInt = numeric_limits<int>::max();

cout << "\nINTEGER OVERFLOW\n";
cout << "Before: " << maxInt << endl;

// Intentional unsafe operation
int overflow = maxInt + 1;

cout << "After : " << overflow << endl;

// 5. String -> numeric
string input = "999999999999999999999999";

try {
    int value = stoi(input);
```

```

        cout << "\nSTRING -> INT\n";
        cout << "Before: " << input << endl;
        cout << "After : " << value << endl;
    }
    catch (...) {
        cout << "\nString conversion failed or value is out of range."
              << endl;
    }

    return 0;
}

```

Output:-

```

INT -> SHORT
Before: 40000
After : -25536
|
SIGNED -> UNSIGNED
Before: -25
After : 4294967271

FLOAT -> INT
Before: 123.78
After : 123

INTEGER OVERFLOW
Before: 2147483647
After : -2147483648

String conversion failed or value is out of range.

```

5. Safe Implementation

```

#include <iostream>
#include <string>
#include <limits>
#include <optional>

using namespace std;

// Safe int -> short
optional<short> safeIntToShort(int value) {
    if (value < numeric_limits<short>::min() ||
        value > numeric_limits<short>::max()) {
        return nullopt;
    }

    return static_cast<short>(value);
}

// Safe signed -> unsigned
optional<unsigned int> safeSignedToUnsigned(int value) {
    if (value < 0) {
        return nullopt;
    }

    return static_cast<unsigned int>(value);
}

// Safe float -> int
optional<int> safeFloatToInt(float value) {
    if (value < numeric_limits<int>::min() ||
        value > numeric_limits<int>::max()) {
        return nullopt;
    }

    return static_cast<int>(value);
}

// Safe string -> int
optional<int> safeStringToInt(const string& text) {
    try {
        size_t position = 0;
    }

```

```

        long long value = stoll(text, &position);

        // Make sure the complete string was converted
        if (position != text.length()) {
            return nullopt;
        }

        // Check int range
        if (value < numeric_limits<int>::min() ||
            value > numeric_limits<int>::max()) {
            return nullopt;
        }

        return static_cast<int>(value);
    }
    catch (...) {
        return nullopt;
    }
}

int main() {

    cout << "==== SAFE DATA TYPE CONVERSION =====\n";

    // 1. int -> short
    int largeInt = 40000;

    cout << "\nINT -> SHORT\n";
    cout << "Before: " << largeInt << endl;

    auto shortResult = safeIntToShort(largeInt);

    if (shortResult) {
        cout << "After : " << *shortResult << endl;
    } else {
        cout << "ERROR: Value is outside short range.\n";
    }
}

```

```

// 2. signed -> unsigned
int negative = -25;

cout << "\nSIGNED -> UNSIGNED\n";
cout << "Before: " << negative << endl;

auto unsignedResult = safeSignedToUnsigned(negative);

if (unsignedResult) {
    cout << "After : " << *unsignedResult << endl;
} else {
    cout << "ERROR: Negative value cannot be converted safely.\n";
}

// 3. float -> int
float decimal = 123.78f;

cout << "\nFLOAT -> INT\n";
cout << "Before: " << decimal << endl;

auto intResult = safeFloatToInt(decimal);

if (intResult) {
    cout << "After : " << *intResult << endl;
} else {
    cout << "ERROR: Value is outside int range.\n";
}

// 4. Check integer overflow before addition
int maxInt = numeric_limits<int>::max();

cout << "\nINTEGER OVERFLOW CHECK\n";
cout << "Before: " << maxInt << endl;

if (maxInt == numeric_limits<int>::max()) {
    cout << "ERROR: Adding 1 would cause signed integer overflow.\n";
} else {

```


SHORT

Before: 40000

```
ERROR: Value is outside short range.
```

-> UNSIGNED

Before: -25

```
ERROR: Negative value cannot be converted safely.
```

Float -> Int

Before: 123.78

After : 123

INTEGER OVERFLOW CHECK

Before: 2147483647

```
ERROR: Adding 1 would cause signed integer overflow.
```

STRING -> INT

Before: 999999999999999999999999

```
ERROR: String contains an invalid or out-of-range integer.
```

7. Result

The experiment demonstrated that unchecked data type conversions can result in truncation, unexpected sign conversion, loss of precision and undefined behavior. The safe implementation prevented invalid conversions by checking the valid range before performing the conversion.

8. Conclusion

Data type conversion should not be treated as automatically safe merely because the C++ compiler allows the conversion. Explicit range checks and appropriate conversion functions can prevent unexpected results and improve software reliability and security